

Módulo de Almacenamiento Farmacológico Automatizado (MAFA)

Dilan Toro Carmona
Santiago Soto Suárez
Oscar Reyes Martinez

Electrónica Digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	3
6. Explicación de funciones principales	4
7. Comunicación	4
8. Interfaz de usuario	4
9. Procedimiento de prueba	5
10. Manejo de errores y seguridad	5
11. Posibles mejoras	5

1. Identificación del proyecto

- **Nombre del proyecto:** Módulo de Almacenamiento Farmacológico Automatizado, MAFA
- **Integrantes del equipo:** Dilan Toro Carmona, Santiago Soto Suárez, Oscar Reyes Martinez.

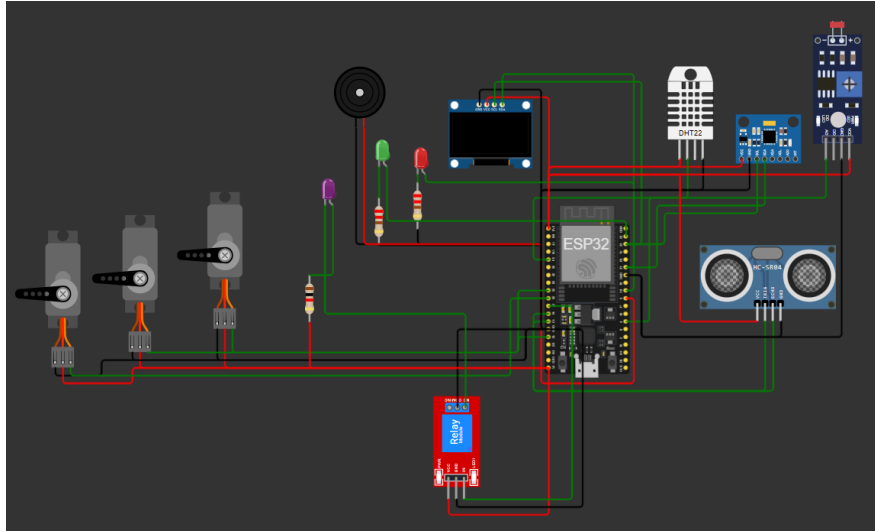
2. Resumen

El proyecto consiste en un sistema de control y monitoreo para un gabinete de almacenamiento de medicamentos. Utiliza un ESP32 para leer sensores de temperatura (DHT11), humedad, movimiento (MPU6050), luz (LDR) y proximidad (ultrasónico). Dependiendo de las condiciones internas (temperatura, humedad, apertura de puerta, movimiento no deseado), el sistema activa indicadores LED, un buzzer de alarma y ejecuta rutinas de apertura/cierre mediante servomotores. Además, implementa un ciclo de desinfección con una lámpara UV-C, envía datos a un servidor remoto y permite la interacción a través de un bot de Telegram.

3. Descripción del hardware

- **Microcontrolador:** ESP32 (modelo con ADC y pines GPIO).
- **Sensor de temperatura y humedad:** DHT11 (pin 4).
- **Sensor de movimiento:** MPU6050 (acelerómetro) en dirección 0x68 (I2C).
- **Sensor de luz:** LDR en pin ADC 34.
- **Sensor de distancia:** HC-SR04 (Trigger en 12, Echo en 14).
- **Actuadores:**
 - **Servomotores:**
 - Cerradura: pin 13.
 - Bisagra 1: pin 25.
 - Bisagra 2: pin 26.
 - **Lámpara UV-C:** controlada por relé en pin 27.
 - **LEDs indicadores:** Rojo (18), Verde (23).
 - **Buzzer pasivo:** pin 5.
- **Pantalla:** OLED SSD1306 (128x64) en I2C (SDA=21, SCL=22).
- **Fuente de alimentación:** 5V (USB o regulada).

Diagrama de conexión:



4. Descripción del software

- **Lenguaje usado:** MicroPython.
- **Librerías:** machine, time, dht, network, urequests, ujson, ssd1306, math.
- **IDE:** Thonny.

Flujo general del programa:

1. Conexión a WiFi y configuración inicial de Telegram.
2. Inicialización de sensores, LEDs, buzzer y servos.
3. En el ciclo principal:
 - a) Leer acelerómetro (MPU6050), luz (LDR), ultrasonido, DHT11.
 - b) Evaluar condiciones de alerta (temp, humedad, luz, movimiento).
 - c) Controlar lógica de apertura/cierre de puerta y desinfección UV-C.
 - d) Actualizar indicadores LED, buzzer y pantalla OLED.
 - e) Enviar datos al servidor y revisar comandos de Telegram.
4. Repetir cada 2 segundos.

5. Estructura del código

- Archivo principal: `main.py`.
- El código es secuencial, con funciones definidas para control de servos, comunicación Telegram y envío de datos.
- Manejo de interrupciones por tiempo para control de estados (puerta abierta, desinfección, alertas).

6. Explicación de funciones principales

- `leer_16bits(reg)`: Lee un registro de 16 bits del MPU6050 y ajusta el valor para números negativos.
- `mover_servo(servo, angulo)`: Controla un servomotor mapeando un ángulo a ancho de pulso.
- `mover_suave` y `mover_sincrono`: Movimientos coordinados y suaves de los servos para abrir/cerrar.
- `abrir_sistema()` y `cerrar_sistema()`: Secuencias completas de apertura y cierre del gabinete.
- `enviar_datos()`: Envía un JSON con temperatura, humedad, movimiento, luz y distancia al servidor.
- `enviar_telegram()` y `enviar_alerta_telegram()`: Notifican por Telegram con límite de frecuencia para evitar spam.
- `recibir_mensajes()` y `responder_comando()`: Escuchan y responden comandos como `/temp`, `/estado`, `/umbrales`.
- Lógica principal en el `while True`: Manejo de estados de la puerta, desinfección UV, y decisiones de alarma.

7. Comunicación

- **Conexión WiFi**: Se conecta a una red WiFi mediante su nombre y contraseña.
- **Servidor**: Envía datos vía POST a `http://10.181.90.208:5000/ingest` con clave API `iot123`.
- **Telegram**: Se conecta a un bot mediante un token y un chat ID por el cual responde comandos y envía alertas.
- **Protocolos**: HTTP (`urequests`), Telegram Bot API.

8. Interfaz de usuario

- **Pantalla OLED**: Muestra en tiempo real:
 - Estado del relé (UV-C).
 - Estado de la puerta (Abierto/Cerrado).
 - Temperatura y su estado (Normal/Caliente/Frío).
 - Humedad y estado (Normal/Alta/Baja).

- Porcentaje de luz y estado.
- Magnitud del movimiento y estado (Reposo/Normal/Alerta).
- **LEDs y Buzzer:** Indicadores visuales y auditivos de alerta.
- **Telegram:** Interfaz remota por mensajes.

9. Procedimiento de prueba

1. Cargar el código `main.py` al ESP32 usando Thonny.
2. Abrir la consola serial para ver mensajes de depuración y la IP asignada.
3. Probar la apertura de la puerta acercando un objeto al sensor ultrasónico (distancia menor a 10 cm).
4. Verificar que los servos abran la puerta y que después de 5 segundos se cierre sola.
5. Observar la cuenta regresiva de 3 segundos y el encendido del relé (lámpara UV-C) por 3 segundos.
6. Forzar condiciones de alerta (calentar el DHT11, mover el MPU6050 bruscamente, iluminar el LDR) y comprobar LED rojo + buzzer + mensaje a Telegram.
7. Enviar comandos desde Telegram (`/temp`, `/estado`, etc.) y verificar respuestas.

10. Manejo de errores y seguridad

- El sensor DHT11 puede fallar en la lectura; se usa un `try-except` implícito al medir.
- El envío a servidor y Telegram tiene manejos de excepción para evitar que el programa se detenga.
- Se evita saturar Telegram con un intervalo mínimo de 60 segundos entre alertas del mismo tipo.
- El sistema tiene lógica de estados para que la desinfección UV-C solo ocurra con la puerta cerrada, evitando exposición.
- No se implementa watchdog, pero el ciclo principal es resiliente a fallos de comunicación.

11. Posibles mejoras

- Agregar un sistema de autenticación para comandos de Telegram.
- Implementar almacenamiento local de logs en caso de pérdida de WiFi.
- Calibrar los rangos del acelerómetro según la orientación real del gabinete.